

LEVEL 19

Chapter 22: Search Systems

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 22: Search Systems

← [Chapter 21: Real-Time Systems](#) · [Index](#) · Next → [Chapter 23: Performance Engineering](#)

You already run Elasticsearch — this chapter converts that operational familiarity into the "why does this work" depth interviewers probe for, since "I'd use Elasticsearch" alone is exactly the kind of unjustified name-drop Chapter 14.4 flags as a mistake.

22.1 The Inverted Index — Why Search Engines Aren't Just "a Database with a WHERE LIKE '%...%'"

The problem with a regular database for text search: `WHERE description LIKE '%wireless headphones%'` forces a full scan of every row's text, can't rank results by relevance, and can't handle "find documents matching *any* of these words, ranked by how well they match" at all — this is a fundamentally different query shape than what B-Tree indexes (Chapter 5.1) are built for.

The inverted index solves this by flipping the data structure: instead of "document → words it contains" (a forward index — which is what a normal row/document naturally is), build "word → list of documents containing it" (inverted). Searching for "wireless headphones" becomes: look up "wireless" → get its document list; look up "headphones" → get its document list; intersect/combine them — a fast, direct lookup instead of a scan, using exactly the same underlying principle as a B-Tree index (Chapter 5.1), just inverted onto words instead of row values.

Tokenization: before building the index, text is broken into searchable units ("tokens") — splitting on whitespace/punctuation, lowercasing ("Wireless" and "wireless" should match), removing common **stop words** ("the," "a," "is" — words too common to be useful signal, though modern search sometimes keeps them for phrase matching), and **stemming/lemmatization** (reducing "running," "runs," "ran" to a common root "run" so a search for one matches documents containing any form) — this pipeline (called an **analyzer** in Elasticsearch, which you've configured yourself) is what determines what actually gets indexed and is searchable, and getting it wrong (e.g., overly aggressive stemming) is a common, real source of "why doesn't my search find this obviously relevant result" bugs.

22.2 Ranking — Why Results Have an Order at All

A match isn't binary — some documents match "better" than others, and ranking is what determines result order. The classic, foundational scoring approach worth being able to explain:

TF-IDF (Term Frequency-Inverse Document Frequency):

- **Term Frequency:** how often the search term appears *in this specific document* — appearing more often suggests more relevance to that document.
- **Inverse Document Frequency:** how rare the term is *across all documents* — a term appearing in almost every document (like "the") carries little discriminating signal, so it's down-weighted; a rare term appearing in a document is a much stronger relevance signal.
- Combined: a document scores highly for a term if that term appears frequently *in it* but is *rare overall* — this is the classic, intuitive foundation, and Elasticsearch's default modern scoring algorithm, **BM25**, is a refined evolution of this same TF-IDF idea (accounting additionally for document length, so a term appearing 3 times in a short document scores differently than 3 times in a very long one).

Beyond pure text relevance — business-signal-blended ranking: as [Chapter 18, Problem 25](#) covered for e-commerce specifically, real production search almost never ranks by pure text relevance alone — it blends in signals like popularity/sales velocity, recency, in-stock status, and (where applicable) sponsored placement, typically as a weighted combination layered on top of the base text-relevance score, not a replacement for it.

22.3 Autocomplete and Fuzzy Search

Autocomplete ("search-as-you-type"): typically implemented with a dedicated structure optimized for prefix matching — an **edge n-gram** analyzer (indexing "headphones" as "h", "he", "hea", "head"... so a partial prefix query matches directly) or a **trie/prefix-tree** structure, rather than running a full inverted-index query against a partial word on every keystroke, which would be both slower and lower-quality (a partial word doesn't have the same statistical properties TF-IDF/BM25 relies on). Autocomplete is also almost always backed by a much smaller, curated dataset (popular queries, product titles) rather than the full document corpus, and heavily cached given how repetitive prefix queries are ("i", "ip", "iph", "ipho"... across millions of users typing the same popular terms).

Fuzzy search (typo tolerance): matching "wireles headphnoes" to "wireless headphones" despite the misspellings — implemented via **edit distance** (Levenshtein distance — the minimum number of character insertions/deletions/substitutions needed to turn one string into another), where a search term is matched against index terms within a small edit-distance threshold (typically 1-2 edits), rather than exact string matching — Elasticsearch's `fuzziness` parameter exposes exactly this.

22.4 Filtering and Faceting

Filtering: narrowing results by exact criteria (category = "electronics", price between X and Y, in_stock = true) — unlike relevance-scored text matching, filters are typically boolean (match or don't) and, in Elasticsearch specifically, filter clauses are **cacheable and don't contribute to the relevance score computation**, which makes them meaningfully cheaper than an equivalent "must match" text query — worth knowing this distinction precisely, since using a `filter` context instead of a `must` (query) context for non-scored criteria is a real, concrete performance optimization, not just a stylistic choice.

Faceting (aggregations): computing "how many results fall into each category" (e.g., "Electronics (450), Clothing (230), Books (89)" shown as filter option counts) — done via Elasticsearch's aggregation framework, computed *alongside* the main search query in the same request rather than as N separate count queries, which would be far more expensive at scale.

22.5 Sharding, Replication, and the Indexing Pipeline

Sharding: an Elasticsearch index is split into multiple **shards**, distributed across nodes — this is the same horizontal-partitioning principle from Chapter 5.4, applied to a search index specifically, allowing both storage and query load to scale across many machines. A search query fans out to all relevant shards and merges results — worth knowing that shard count is set at index-creation time and is expensive to change later (a real operational gotcha, and a legitimate thing to mention if asked about capacity planning for search).

Replication: each shard is replicated across nodes for both durability and **read throughput** (replica shards can serve search queries too, not just stand by for failover) — directly analogous to database read replicas (Chapter 5.3).

The indexing pipeline — keeping search in sync with the source of truth: search indices are almost always a **derived, secondary data store**, not the system of record — the actual product/document data lives in a primary database, and search is populated via an indexing pipeline, commonly using the **CDC pattern** (Chapter 6.8) to stream changes from the primary DB into Elasticsearch asynchronously. This means search is **deliberately eventually consistent** with the source of truth — a newly created product might take a few seconds to become searchable, which is an acceptable, well-understood trade-off (echoing [Chapter 18, Problem 25](#)'s explicit framing), not an oversight.

Interview question: "Your product database and your Elasticsearch index show different data for a product that was just updated. Is this a bug?"

Ideal senior answer: "Not necessarily — search is a derived index kept in sync asynchronously, typically via CDC from the primary database, so a brief lag between a write landing in the source of truth and it becoming visible in search is expected, not a correctness bug. What *would* be a real bug is if that lag grew unbounded (the indexing pipeline falling behind and never catching up) or if the two permanently diverged rather than eventually converging — I'd monitor indexing-pipeline lag as a specific metric precisely to catch that failure mode, the same way I'd monitor replication lag for a database read replica."

22.6 Applying This: Job Search

A quick, distinct application worth naming since it has a genuinely different ranking problem than e-commerce: job search ranking blends text relevance (title/skills match) with **recency** (a 2-day-old posting usually should outrank a functionally-identical 60-day-old one, given job postings have a natural freshness decay that products generally don't) and, often, a **two-sided relevance signal** — not just "does this job match the searcher's query" but a personalization layer considering the searcher's own profile/history, which starts to blend into the Recommendation System territory from [Chapter 18, Problem 26](#) — worth explicitly naming that overlap if the interview heads that direction, rather than treating job search as a purely standalone text-search problem.

Chapter 22 Interview Drill

1. Explain the inverted index and why it's structurally different from a B-Tree index, in your own words.
2. Explain TF-IDF/BM25 at a level you could defend if challenged on "why does document A rank above document B."
3. Why is autocomplete typically backed by a separate, smaller data structure than the main search index?
4. Explain the performance difference between Elasticsearch's **filter** and **query** (must) contexts.
5. Why is a search index almost always eventually consistent with its source-of-truth database, and what metric would you monitor to catch that lag becoming a real problem?

Next → [Chapter 23: Performance Engineering](#) — identifying and fixing bottlenecks across CPU, memory, network, database, and cache layers.